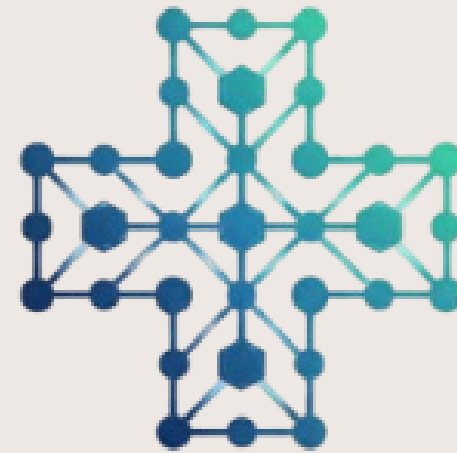




RedNorte

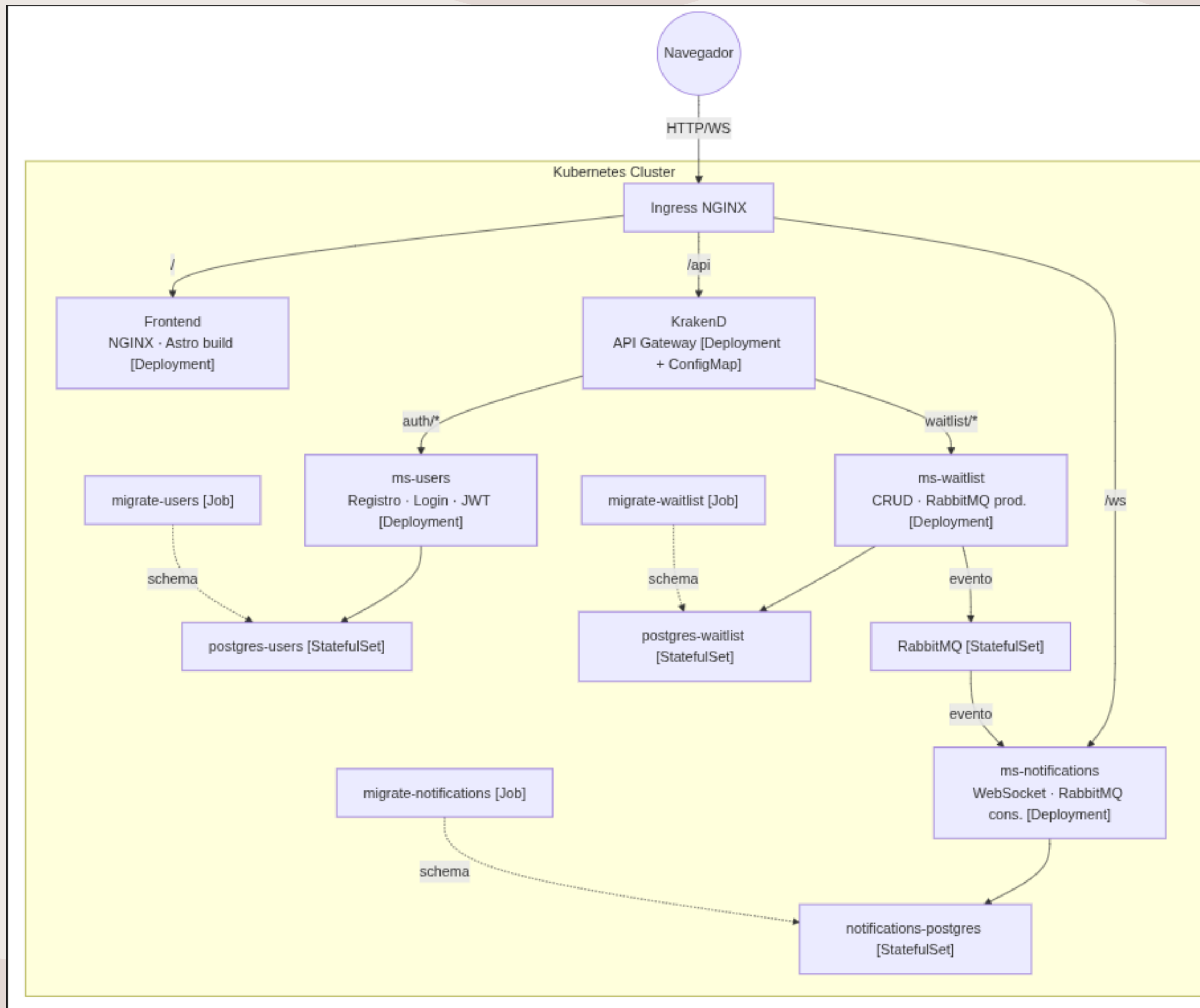
Servicio Público de Salud — Plataforma Inteligente para la Gestión de Listas de Espera

Integrantes: Sebastián Godoy
Docente: Claudio Rojas
Asignatura: Desarrollo Fullstack III
Sección: 003D

ÍNDICE

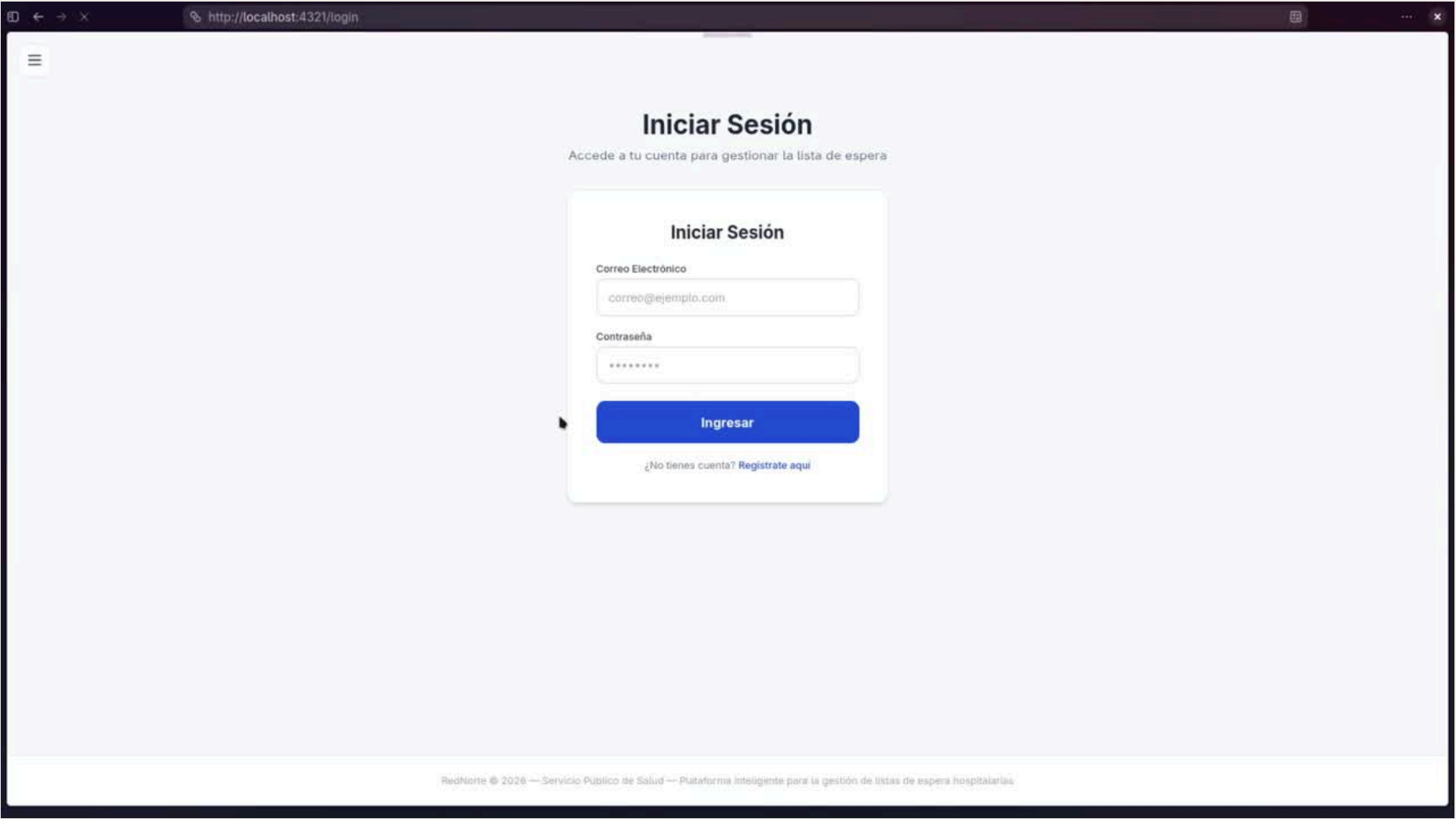
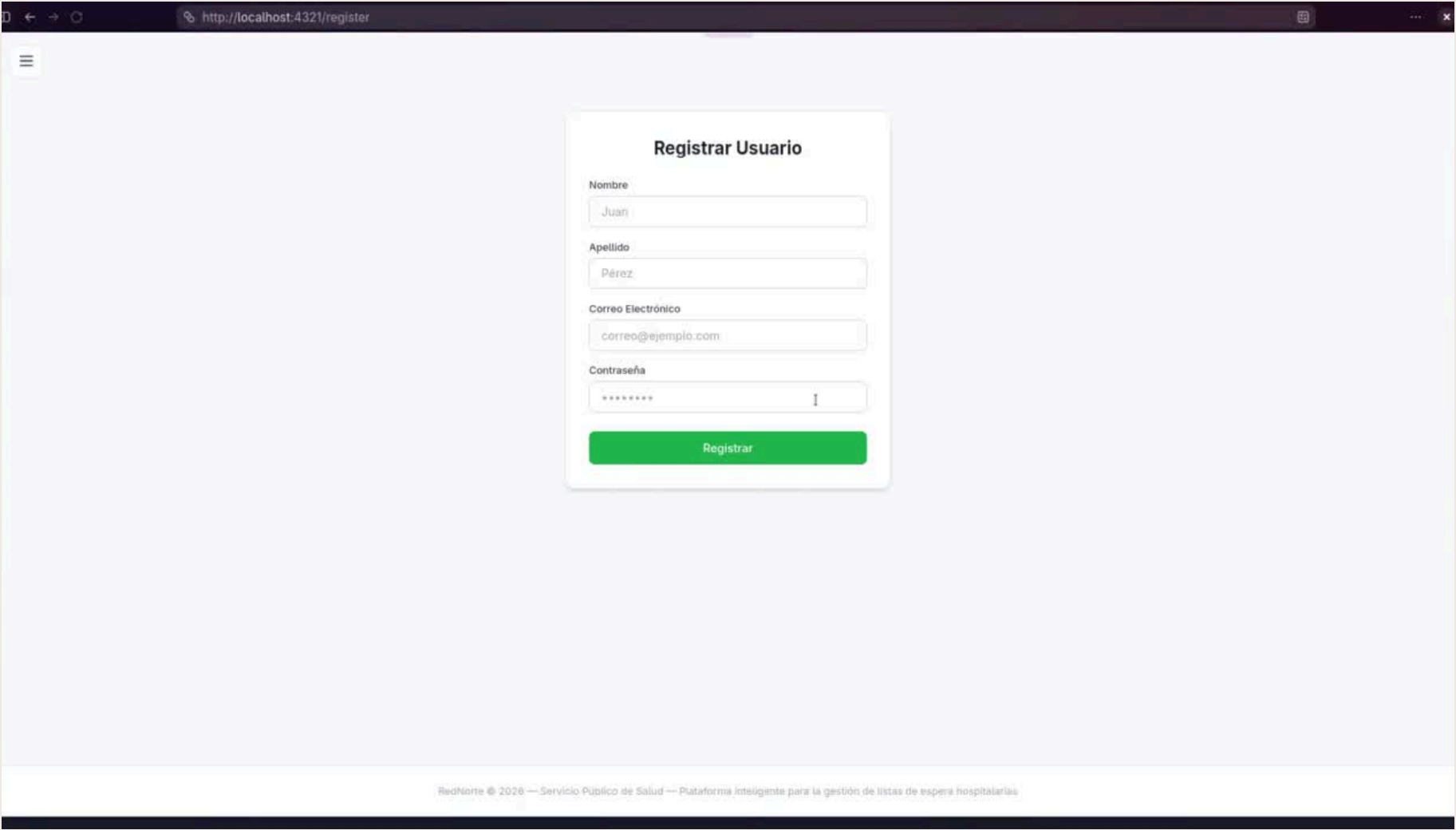
- Arquitectura del Sistema
- Demostración
- Patrones de Diseño
- Estrategia de Branching
- Pruebas Unitarias

ARQUITECTURA DEL SISTEMA





DEMOSTRACION





RedNorte

Servicio Público de Salud

Plataforma inteligente para la gestión de listas de espera hospitalarias. Optimizamos la administración de pacientes, priorizamos casos críticos y facilitamos la comunicación entre el personal médico y los pacientes.

Conocer más

Comenzar ahora

Sobre RedNorte

El Servicio Público de Salud RedNorte administra una red de hospitales, centros de atención primaria y clínicas especializadas en el norte del país. Cada año enfrenta el desafío de gestionar eficientemente las listas de espera para consultas, procedimientos y cirugías.

Esta plataforma inteligente de priorización asegura que los casos más urgentes reciban atención inmediata, optimiza el flujo de pacientes y maximiza la eficiencia del personal médico en toda la red.

PATRONES DE DISEÑO

1

```
1 async findByEmail(email: string) {  
2     const result = await sql`SELECT * FROM users WHERE email = ${email}`;  
3  
4     return result.length > 0 ? result[0] : null;  
5 }
```

repositories
user_repository.test.ts
TS user_repository.ts

2

```
1 async addPatientToWaitlist(  
2     data: Pick<WaitlistEntry, "userId" | "priority" | "reason">,  
3 ) {  
4     // Verifica que la prioridad este dentro del rango existente  
5     if (data.priority < 1 || data.priority > 4) {  
6         throw new AppError("Nivel de prioridad inexistente");  
7     }  
8 }
```

services
waitlist_service.test.ts
TS waitlist_service.ts

3

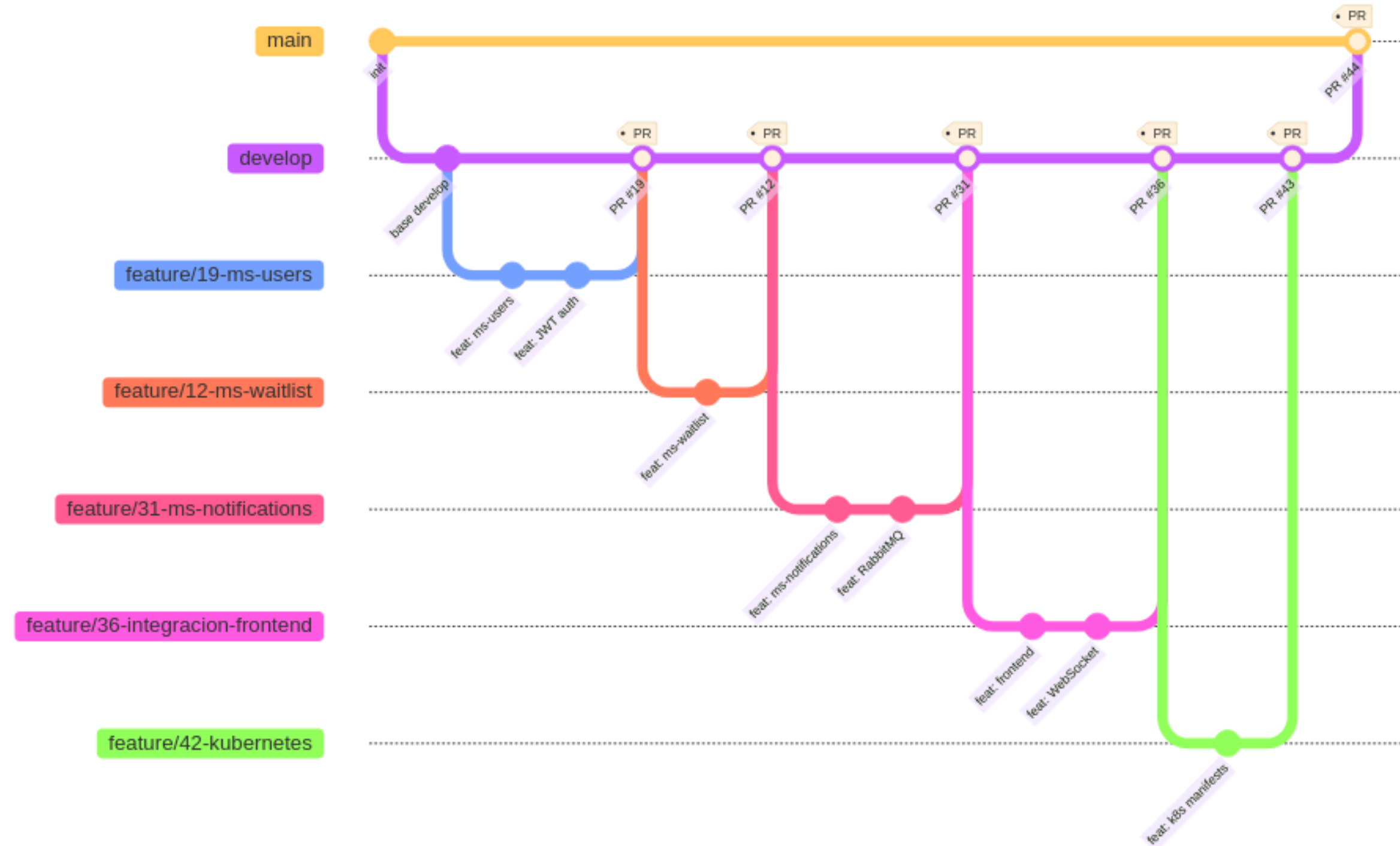
```
1 export function errorHandler(err: Error, c: Context) {  
2     const status = err instanceof AppError ? err.statusCode : 500;  
3     console.error(`[Error] ${status} - ${err.message}`);  
4     return c.json({ success: false, message: err.message }, status  
5     as any);  
6 }
```

lib
TS app-error.ts
TS error-handler.ts



ESTRATEGIA DE **BRANCHING**

Estrategia de Branching — Git Flow



Leyenda

- **main** — Producción, estable
- **feature/*** — Nueva funcionalidad
- **fix/*** — Corrección de bugs

- **develop** — Integración
- **chore/*** — Mantenimiento
- **PR** — Pull Request aprobado

PRUEBAS UNITARIAS

Componente	Tests	Cobertura Líneas
ms-users	19	98.43%
ms-waitlist	19	94.67%
ms-notifications	20	99.35%
Frontend	36	97.97%
Total	94	=>94%

CONCLUSIÓN

- **Arquitectura de microservicios con aislamiento de datos**
- **Notificaciones en tiempo real (WebSocket + RabbitMQ)**
- **94 tests, cobertura =>94%**
- **Despliegue en Docker Compose y Kubernetes**

MUCHAS GRACIAS